

07 dijkstra

Dijkstra's Algorithm

1. What is it, and when is it used?

Dijkstra's algorithm finds the shortest path from a single source node to all other nodes in a weighted graph with **non-negative edge weights**. It's used whenever you need "cheapest/shortest path" in a graph where edges have costs/weights, and none of those costs are negative.

2. Why is it used?

BFS gives shortest paths only when all edges have equal weight (unweighted graphs). The moment edges have different positive costs, BFS's "layer by layer" guarantee breaks down. Dijkstra's greedily always expands the currently-cheapest-known node next, using a min-heap, guaranteeing that once a node is popped, its shortest distance is finalized — giving correct shortest paths in $O((V+E) \log V)$.

3. Where is it used?

- GPS / map navigation (cheapest route by distance or time)
- Network routing protocols (OSPF uses Dijkstra internally)
- Flight/train ticket cheapest itinerary problems
- "Cheapest flights within K stops" (modified Dijkstra or Bellman-Ford)
- Game pathfinding with weighted terrain costs
- Network delay time / time for signal to reach all nodes
- Any "minimum cost to reach every node from a source" with non-negative weights

4. How do you identify it's a Dijkstra problem?

Signal phrases (2-minute test): - "shortest path" / "minimum cost path" / "cheapest route" with **weighted edges** - "time for signal to reach all nodes" (weighted graph) - "minimum effort path" / "path with minimum maximum edge" (Dijkstra variant using different relaxation) - Weights represent distance, time, or cost, and are explicitly **non-negative** - Single source, need distances to all (or one specific) other nodes

5. Can you identify it from constraints?

Constraint clue	Implication
Edge weights explicitly stated as non-negative (e.g., <code>1 <= weight <= 1000</code>)	Dijkstra is safe and optimal
Edge weights can be negative	Dijkstra breaks — must use Bellman-Ford (or Bellman-Ford + Johnson's for all-pairs)
Graph unweighted, or all weights equal	Plain BFS is simpler and faster — no need for Dijkstra
V, E up to 10^5 , need shortest path from 1 source	Dijkstra with binary heap: $O((V+E) \log V)$ — fits comfortably
Need shortest path between all pairs	Consider Floyd-Warshall (dense, small V) or run Dijkstra from every node (sparse, large V)
"At most K stops/edges" constraint added	Standard Dijkstra doesn't directly track hop count — needs state augmentation (distance, stops) or Bellman-Ford style relaxation for K rounds

6. Types / Variants

1. **Dijkstra with Binary Heap (PriorityQueue)** — standard, $O((V+E) \log V)$.
2. **Dijkstra with Fibonacci Heap** — theoretically $O(E + V \log V)$, rarely implemented in interviews.
3. **Dijkstra with array (no heap)** — $O(V^2)$, better for dense graphs (E close to V^2).
4. **Dijkstra variant: minimize the maximum edge weight on a path** (“minimum effort path”) — same greedy relaxation, different comparison function.
5. **Multi-source Dijkstra** — push all sources into the heap initially with distance 0.
6. **Dijkstra with state augmentation** — e.g., track (node, stops_used) as the “node” in the heap for constrained shortest path problems.

7. Rationale / Intuition

Dijkstra’s greedy correctness relies entirely on **non-negative weights**: once you pop the node with the smallest tentative distance from the priority queue, you can be sure no other path could ever produce a smaller distance to it — because all other paths would have to go through an edge of weight ≥ 0 , which can only add to a distance, never subtract. This greedy “finalize the closest frontier node” strategy is exactly why it fails with negative weights: a later negative edge could retroactively make an already-finalized path cheaper, but Dijkstra never revisits finalized nodes.

8. Time & Space Complexity

Implementation	Time	Space
Binary heap (PriorityQueue)	$O((V + E) \log V)$	$O(V + E)$
Array-based (no heap), dense graph	$O(V^2)$	$O(V + E)$
Fibonacci heap (theoretical)	$O(E + V \log V)$	$O(V + E)$

9. Comparison with Other Patterns

Algorithm	Handles negative weights?	Complexity	Use case
BFS	N/A (unweighted only)	$O(V + E)$	Unweighted shortest path
Dijkstra	No	$O((V+E) \log V)$	Single-source shortest path, non-negative weights
Bellman-Ford	Yes (also detects negative cycles)	$O(V \cdot E)$	Single-source, negative weights allowed
Floyd-Warshall	Yes (no negative cycles)	$O(V^3)$	All-pairs shortest path, small dense graphs
A*	No (like Dijkstra + heuristic)	Depends on heuristic	Single-source-to-single-target with a good heuristic (faster in practice)
Union-Find	N/A	$\sim O(1)$ amortized	Connectivity, not path length

Decision rule: unweighted $\rightarrow$ BFS. Weighted, non-negative, single source $\rightarrow$ Dijkstra. Weighted, negative edges allowed $\rightarrow$ Bellman-Ford. All-pairs, small V ($\leq \sim 400$ -500) $\rightarrow$ Floyd-Warshall.

10. Reusable Java Templates

10.1 Dijkstra — Single Source Shortest Path (Adjacency List + PriorityQueue)

```
import java.util.*;

class Dijkstra {
    public int[] shortestPath(int n, List<int[]>[] adj, int src) {
        // adj[u] contains int[]{v, weight}
        int[] dist = new int[n];
```

```

Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;

PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.offer(new int[]{src, 0}); // {node, distance}
boolean[] visited = new boolean[n];

while (!pq.isEmpty()) {
    int[] curr = pq.poll();
    int u = curr[0];
    if (visited[u]) continue;
    visited[u] = true;

    for (int[] edge : adj[u]) {
        int v = edge[0], weight = edge[1];
        if (!visited[v] && dist[u] != Integer.MAX_VALUE
            && dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            pq.offer(new int[]{v, dist[v]});
        }
    }
}

return dist;
}

// Helper to build adjacency list from edge list
@SuppressWarnings("unchecked")
public List<int[]>[] buildGraph(int n, int[][] edges, boolean directed) {
    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] e : edges) {
        adj[e[0]].add(new int[]{e[1], e[2]});
        if (!directed) adj[e[1]].add(new int[]{e[0], e[2]});
    }
    return adj;
}
}

```

10.2 Dijkstra Variant — Minimum Effort Path (minimize max edge on path)

```

import java.util.*;

class MinimumEffortPath {
    public int minimumEffortPath(int[][] heights) {
        int rows = heights.length, cols = heights[0].length;
        int[][] effort = new int[rows][cols];
        for (int[] row : effort) Arrays.fill(row, Integer.MAX_VALUE);
        effort[0][0] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]);
        pq.offer(new int[]{0, 0, 0}); // {row, col, effort}
        int[][] dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int r = curr[0], c = curr[1], e = curr[2];
            if (r == rows - 1 && c == cols - 1) return e;
            if (e > effort[r][c]) continue;

            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
                int newEffort = Math.max(e, Math.abs(heights[nr][nc] - heights[r][c]));
                if (newEffort < effort[nr][nc]) {

```

```
        effort[nr][nc] = newEffort;
        pq.offer(new int[]{nr, nc, newEffort});
    }
}
return 0;
}
```

11. Quick Recognition Checklist (2-minute test)

- ☐ Weighted graph, weights ≥ 0 ?
- ☐ "Shortest path" / "cheapest cost" / "minimum time" from a single source?
- ☐ Not asked about negative edges or negative cycle detection?
- ☐ Not asking for all-pairs distances on a small dense graph (that's Floyd-Warshall)?

If 3+ checked → Dijkstra's Algorithm.